

Software Architecture and Platforms
-
7th Assignment

Simone Ceredi

August 19, 2026

Contents

1	Analysis	2
1.1	Requirements	2
1.2	Analysis	2
1.2.1	Health check API	2
1.2.2	Application metrics	3
1.2.3	Distributed logging	3
1.2.4	Distributed tracing	3
2	Architecture	4
2.1	Application metrics	4
2.2	Distributed logging	4
2.3	Distributed tracing	4
3	Detailed Design and Implementation	5
3.1	Health check API	5
3.2	Application metrics	5
3.3	Distributed logging	6
3.4	Distributed tracing	6
4	Conclusions	7

Chapter 1

Analysis

The seventh assignment for the Software Architecture and Platforms course is the application of patterns that make the microservices developed in the fifth assignment production ready.

1.1 Requirements

Based on the solution developed for the fifth assignment or the cooperative pixel art example, the objective is to apply the following patterns:

- health check API;
- application metrics;
- distributed tracing;
- distributed logging.

1.2 Analysis

I have decided to use the solution developed for the fifth assignment as a starting point as it does contain microservices written in Go, Kotlin and an API gateway that uses spring, for this reason I will be able to explore how the patterns can be implemented in different languages.

1.2.1 Health check API

The health check API pattern is used to know the state of a service. To achieve this an endpoint is exposed, usually at `/health` or `/healthz` that returns 200 if everything is working correctly.

It is important to define what a healthy service is, as being able to respond to a request usually isn't enough. In order to be in a healthy state a service should also check that the resources that it uses, such as databases or other permanent external connections are working.

1.2.2 Application metrics

Application metrics are a key part of what makes a system production ready. They allow administrators to see how the system is progressing and how its working at a glance, by tracking useful metrics. Those metrics may be infrastructure level, such as CPU, memory or disk utilization, application level metrics as request latency and number of requests executed or even domain specific metrics that allow to see if the system is really allowing the users to do what is required, for example in the case of a shopping site the amount of products that have been bought in a given day.

All those metrics allow the development of an alert system, which may alert the system maintainers when some metrics are outside the predefined ranges. Moreover, domain specific metrics may be used for market analysis or other types of data analysis efforts.

1.2.3 Distributed logging

Distributed logging is a fundamental component of microservice systems. In monolithic architectures the logs all reside in a file or the standard output of the application, while in microservice ones they are scattered on different services that may be managed via Docker or Kubernetes, making them more difficult to access.

Distributed logging consists in aggregating those log sources in a single place. It usually also includes a way to search and filter the logs. This way it is easier to debug the system while it is running, or extract information from the different microservices interacting.

1.2.4 Distributed tracing

Distributed tracing allows a user to see how a request moves across the microservices, it goes on to show a flame graph similar to what performance profilers do for monolithic applications. To do so, an identifier is added to a request when it passes the API gateway, and every service, upon receiving the request and sending the response, sends some information to the distributed tracing server. Via the distributed tracing server it is then possible to observe and analyze the results.

Chapter 2

Architecture

The system architecture needs to be updated to make room for the infrastructure services needed to implement the four patterns.

2.1 Application metrics

Application metrics are managed via Grafana and Prometheus so both services need to be added to the infrastructure. Grafana is located in a network that only contains its service and the Prometheus service, while the latter is also inside the networks of each service as the services need to contact its endpoint to upload the metrics.

2.2 Distributed logging

Distributed logging is implemented via the ELK stack. ELK is composed of Elasticsearch, Logstash and Kibana so all three components need to be added to the system. Logstash can collect logs in a variety of methods, but the simplest to implement in this case is via log files. This means that a volume needs to be created to collect the log files of all services.

2.3 Distributed tracing

Distributed tracing would have been implemented via Zipkin. This would have meant adding the component to the infrastructure and allowing each service to contact it with the request information. Unfortunately as seen in section 3.4 introducing it in a heterogeneous system as this is not simple.

Chapter 3

Detailed Design and Implementation

3.1 Health check API

Health checks have been implemented in all microservices. Each service checks the connection to its database, as the system uses the database per microservice pattern, using a ping, and upon receiving a positive response from the database returns a 200 response, a 503 otherwise.

This health check endpoint is also added to the Dockerfile used to build the service so that when running the service it is also possible to see its state and allows docker to restart the service if needed.

The same solution is implemented in every service, while rides, users and e-scooter services are implemented by hand, for the API gateway there is already a health check actuator managed by spring, so only a route redirection is needed, so that requests to `/health` are redirected to `/actuator/health`.

3.2 Application metrics

The application metrics server is implemented using Grafana and Prometheus. The metrics follow a strict contract regarding the format of those so while it is recommended to use a library to send them, it is also possible to do it without the need for an external dependency.

While in the API gateway the metrics are managed via the Micrometer library, the other services are using a custom metric that counts the total amount of requests received by the service and are sent using plain http responses.

3.3 Distributed logging

Using the ELK stack it is possible to define different methods to pick up logs and save them in the Elasticsearch database. The simplest solution for this project is the use of a log file that both the microservice and Logstash have access to.

Achieving this also means that the external configuration pattern has to be introduced in the microservices, to allow the specification of which log file to use. On each service it is necessary to specify the path on which the log file will be saved via the `LOG_FILE` environment variable. This way all logs are redirected to that file and it is trivial to configure Logstash to look at those files. Without the external configuration every microservice could have used a different location or standard to define where to save the logs making the distributed logging system more brittle.

3.4 Distributed tracing

Adding distributed tracing to the system is the implementation that was taking the longest and in the end was abandoned.

To add the pattern Zipkin would have needed to be added to the system, and it could have also reused Elasticsearch as a database. Then the API gateway would have needed to be configured, and this is where using a framework like spring should have made things easier, but due to compatibility problems and a lack of documentation what was described to be the configuration needed was not working. After the API gateway each microservice had to be configured, the Users and Rides microservices could have used Vert.x directly, as it can be configured to work seamlessly with Zipkin, while the E-scooters service, implemented in Go, would have needed the use of the OpenTelemetry SDK.

Unfortunately due to the problems encountered during the addition of the pattern on the API gateway no further step has taken place and no distributed tracing has been implemented in the system.

Chapter 4

Conclusions

Going from a microservices system to one that is ready to be deployed in production is not an easy step, a lot of components and practices have to be taken into consideration.

All the patterns implemented in the assignment should, in a real project, be implemented from the start of the implementation process as they are not only useful when the system is in production, but also during development. Health checks allow to see at a glance what microservices are crashing, distributed logging would then be useful to identify the reasons for those problems. Distributed tracing along with application metrics can also be useful during development to identify bottlenecks and other performance issues among other things.

One problem with the solution is the impossibility to add distributed tracing using Zipkin to the system, while usually using frameworks such as spring speeds up the development process, as it was the case this time as well up until this point, when there are incompatibilities or not well documented procedures it becomes a lot more difficult to find a solution with respect to developing a service and only using the few libraries that are strictly needed.